Intro to Ai Lab 1: 8-Puzzle Solver

Team members:

Adham Mohamed Alshabasy id : 23010270
Ahmed Yousry Ahmed id : 23010266

1 Data Structures Used

To efficiently model the 8-puzzle state space and execute the search algorithms, the following data structures were utilized:

- **Node Class (Dataclass):** Acts as a wrapper for each state. It stores the current board configuration (as a 1D tuple), a reference to the parent Node to trace the path back to the initial state, the action taken to reach it, the path cost so far (g), and the total estimated cost ($f = g + h$).
- **Tuples:** The 3x3 board is flattened into a 9-element tuple. Tuples are immutable and hashable, allowing them to be safely and efficiently stored in sets.
- **Sets (Hash Maps):** An `explored` (or `reached`) set is used across the algorithms to store visited states in $O(1)$ lookup time, preventing infinite loops and redundant state expansions.
- **Deque (Double-Ended Queue):** Used in Breadth-First Search (BFS) to implement a First-In-First-Out (FIFO) queue, allowing $O(1)$ appends and pops from the left.
- **List (as a Stack):** Used in Depth-First Search (DFS) and Iterative Deepening Search (IDS) as a Last-In-First-Out (LIFO) stack.
- **Min-Heap (Priority Queue):** Implemented via Python's `heapq` module for the A* search. It automatically keeps the node with the lowest f cost at the top for optimal expansion.

2 Search Algorithms Explanation

The agent employs four distinct algorithms to navigate the state space:

- **Breadth-First Search (BFS):** An uninformed algorithm that explores the state space level by level. By using a FIFO queue, BFS guarantees the shortest path to the goal. However, it is highly memory-intensive, as the number of nodes expanded grows exponentially with the depth of the solution.

- **Depth-First Search (DFS):** An uninformed algorithm that dives as deep as possible down a single branch before backtracking, utilizing a LIFO stack. While highly memory-efficient, it is not optimal. As observed in our tests, DFS can easily get trapped exploring tens of thousands of nodes and returning massive, inefficient paths before finding the goal.
- **Iterative Deepening Search (IDS):** An uninformed search that combines the memory efficiency of DFS with the optimality of BFS. It repeatedly executes a depth-limited DFS, increasing the depth limit by 1 each iteration. While it recalculates upper levels multiple times, it prevents the algorithm from getting lost in infinite branches and guarantees the shortest path.
- **A* Search:** An informed search algorithm that uses the evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is the exact cost to reach the node and $h(n)$ is the estimated heuristic cost to the goal. By using a priority queue, A* optimally directs the search toward the goal, drastically reducing the number of nodes expanded compared to uninformed searches.

3 Heuristic Comparison and Admissibility

For the A* algorithm, two heuristics were tested:

1. **Manhattan Distance:** The sum of the absolute horizontal and vertical distances of each tile from its target position.
2. **Euclidean Distance:** The straight-line distance of each tile from its target position.

Conclusion:

- Both heuristics are admissible (they never overestimate the true cost to reach the goal), guaranteeing an optimal path.
- **Manhattan distance is the superior and more informed heuristic** for the 8-puzzle. Because puzzle tiles can only move horizontally and vertically, the Manhattan distance provides an estimate much closer to the actual required number of moves.
- Euclidean distance assumes a straight diagonal line, which slightly underestimates the true cost compared to Manhattan.
- As seen in our benchmark tests, A* Manhattan consistently expands fewer nodes than A* Euclidean to find the exact same optimal path.

4 Assumptions and Extra Work

- **Assumptions:** It is assumed that the blank tile (0) moves to swap positions with adjacent tiles. Valid inputs must be exactly 9 unique digits (0-8) with an even number of inversions to be solvable. We also assume uniform step costs (every move costs exactly 1) and that execution time comparisons are hardware-dependent, making nodes expanded the primary metric for algorithm efficiency.
- **Priority Queue Tie-Breaking:** In A*, when two paths have the exact same f cost, we assume the priority queue breaks ties by picking the state that was generated first, rather than using a secondary heuristic.
- **Extra Work - File Exporter:** To meet the strict requirement that every step must be printed for traceability while preventing massive DFS outputs from freezing the terminal, a custom `export_results_to_file` function was implemented. This interactively prompts the user to save a complete `.txt` report, applying a safety cap to massive paths to prevent file-size bloat.

5 Sample Runs and Benchmark Comparison

Sample Run 1: Moderate Complexity (Cost: 20)

Initial State: `1 2 5 3 4 0 6 7 8` **Goal State:** `0 1 2 3 4 5 6 7 8`

Algorithm	Cost (Moves)	Nodes Expanded	Search Depth	Time (s)
A* Manhattan	20	546	20	0.0038
A* Euclidean	20	675	20	0.0043
BFS	20	41,086	20	0.1243
DFS	85,244	90,185	85,244	0.2640
IDS	20	164,402	20	0.3471

Observation: A* efficiently finds the optimal 20-move path with minimal expansions. BFS and IDS confirm the optimal cost but expand massively more nodes. DFS wanders blindly, resulting in an incredibly inefficient 85,244-move path.

Sample Run 2: High Complexity (Cost: 24)

Initial State: `0 8 7 6 5 4 3 2 1` **Goal State:** `0 1 2 3 4 5 6 7 8`

Algorithm	Cost (Moves)	Nodes Expanded	Search Depth	Time (s)
A* Manhattan	24	544	24	0.0028
A* Euclidean	24	1,297	24	0.0096
BFS	24	133,800	24	0.3013
DFS	10,282	10,521	10,282	0.0307
IDS	24	914,376	24	1.6209

Observation: While DFS randomly stumbled upon a solution quickly in this run, the resulting path is severely sub-optimal (10,282 moves vs the optimal 24). Additionally, A* Manhattan demonstrates its superiority over Euclidean by requiring fewer than half the node expansions.